# Introduction to Bioinformatics) Excercise Guidebook

Javier F. Tabima Restrepo

2022-09-01

# Contents

# Chapter 1

# MBB101/BIOL123 (Introduction to Bioinformatics) Excercise Guidebook

Welcome to *Introduction to Bioinformatics*! This document here will be your **excercise guidebook**, or what some will call the **Laboratory notebook**.

You will find here all the exercises for class, as well as a basic reference guide for basics of coding and all the resources we will use for the course

## Additional information about the professor

Professor Tabima Restrepo is always reachable at `jtabima@clarku.edu`.

Office hours are on *Wednesdays between 1PM and 3PM* at Larsy 223, or by email.

# Chapter 2

# Structure of the Guidebook and FAQs

## 2.1 What do we need for lab sessions?

For our labs we are going to use *R, RStudio* and the basic *UNIX command line* We will also use *atom*, a basic text editor that's widely used in programming to view and edit files.

We will use the local cluster called `SMAUG` (See Figure 1). `SMAUG` is a computer that the Tabima lab hosts that has 32 processors, more than 14 TB of storage and 128 GB of RAM. Enough to do any kind of analyses.

## 2.2 Why are we using these programs?

While most of our stuff will be executed online, I thought it would be great that we learn a bit about programming and reproducible science.

`R` and `RStudio` will allow us to do that. With R we can create code that can be reproducible and we can use for future analisis. With RStudio we will have an easy usable user interface to create notebooks like this one!

`Atom` will help us read files we wouldn't be able to read otherwise. As we go forward with class we will see some examples of these files.

## 2.3 Do we need to know how to program?

No. We will use R for very basic things like extracting sequences from a large file or to count numbers of genes or basic stuff.

We will learn how to do some basic programming and how to use the command line (i.e. how to use basic commands, how to create folders, execute programs and create loops). If not, we will use internet based tools to learn bioinformatics.

*Being recursive and using the resources we can is very important. Sometimes things won't work as expected and we MUST find an alternative. This is one of those cases.*

---

# Chapter 3

# Software for class

As I mentioned in the preovious chaopter, we are going to use *R, RStudio* and the basic *UNIX command line* for our exercises, as well as *atom*, a basic text editor that's widely used in programming to view and edit files.

*For this course we WON'T need to install R or R studio in our computers! We will use a webpage that has R and R studio.* That being said, I'll leave this guide in case you want to install it in the future.

## 3.1 How to install R?

1. Open an internet browser and go to www.r-project.org
2. Click the "download R" link in the middle of the page.

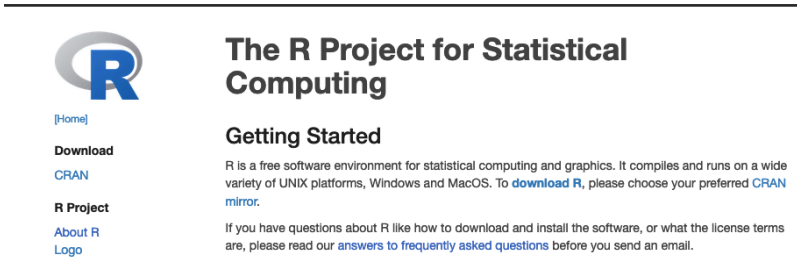



Figure 3.1: Downloading R

3. Select a CRAN location (a mirror site) and click the corresponding link.

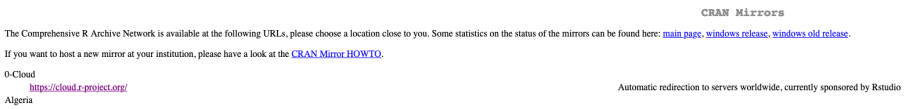



Figure 3.2: Downloading R

4. Click on the "Download R for" your operating system link at the top of the page.

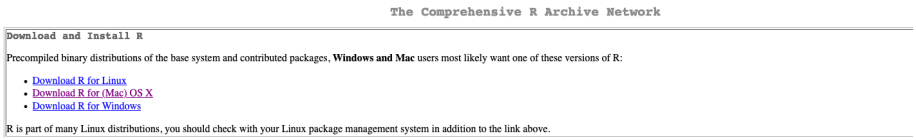


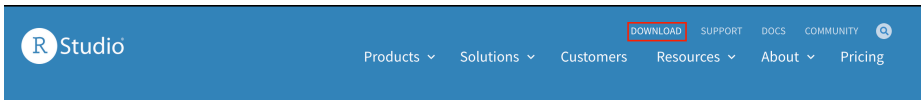

Figure 3.3: Links for R

5.

- For Windows users: Click on the "install R for the first time" link at the top of the page. Run the `.exe` file and follow the installation instructions.
- For MacOS X users: Click on the "R-4.0.2.pkg" link to download the install package. Run the `.pkg` file and follow the installation instructions.

## 3.2 How to install RStudio?

1. Go to www.rstudio.com and click on the "Download" link.




2. Click on "Download RStudio Desktop (FREE)" in the lower part of the page.

3. Click on the version recommended for your system. save the .exe/.dmg file on your computer, double-click it to open, and then drag and drop it to your applications folder.

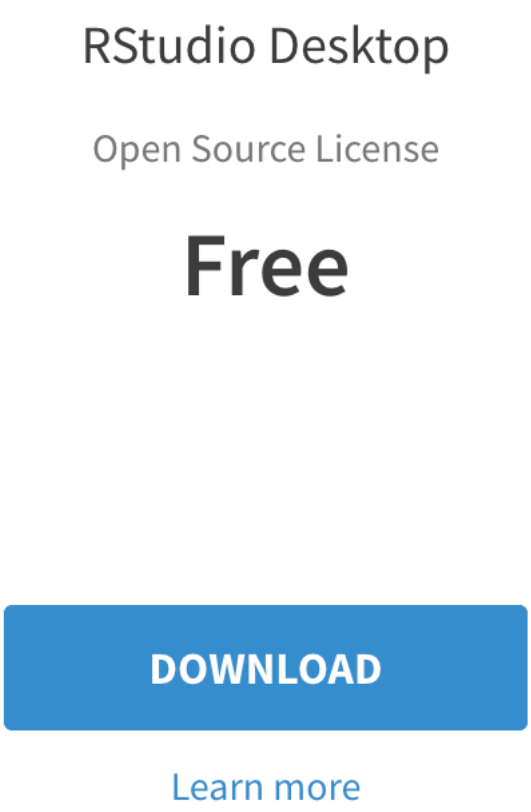



Figure 3.4: Links for R

## 3.3 How to install Atom?

1. Go to the atom webpage at https://atom.io/
2. Click on Download.
3.

- In Mac OS: Move the application to the Applications folder
- In Windows: Execute the `AtomSetup.exe` file

---

# Chapter 4

# Testing the software

## 4.1 Testing R and RStudio

1. Go to http://140.232.222.154:8787
2. Log in using your Clark username *without* the `@clarku.edu` (e.g. my user would be `jtabima`) and the `BIOL123` password all in capitals
3. You should see a window like this:

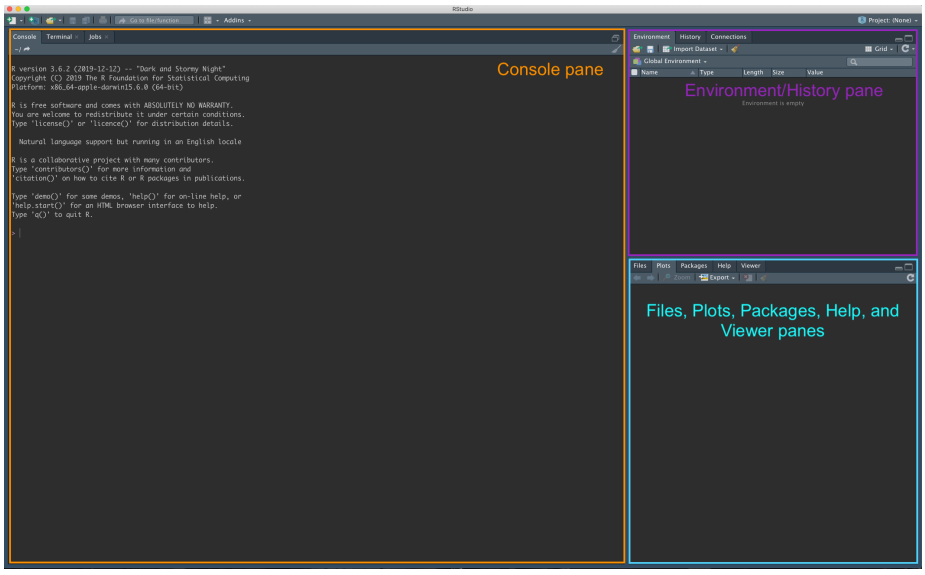



Figure 4.1: RStudio panels

You should have three panes open (or probably four). The one in the left that says `R version XXX` is your *console*. Most of the code goes there, both input and output.

On the upper right you have the *environment/history* pane. The environment stores and shows you all the files being used by the current R session that are saved in your RAM. We will rarely use the environment in this course, but its still an important feature to know how much memory is being used, how many files are loaded and if our objects are actually being used by R. The history panel will show you all the previously used code.

Finally, the lower right has the *Files, Plots, Packages, Help, and Viewer panes*. These panes show exactly that: The files in the folder you are in, the plots generated by R, the R packages that will be loaded, and the help for the different R packages. The Viewer panel is a special feature that is used by some packages, we will use it later.

3. Now, you will see the menu in the top. Click on the `new file` element and lets do our first R notebook.

4. Click on `new file`, and then `R notebook`

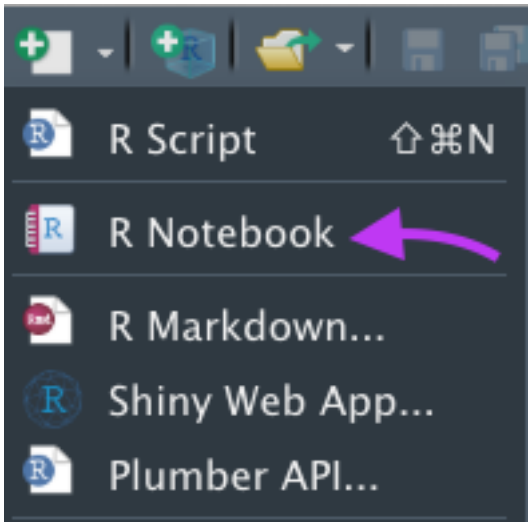



Figure 4.2: RStudio panels

We will talk on class about R notebooks and why they are important for our course!

## 4.2 Testing Atom

1. To test Atom, download the `atom_test` file from the moodle page
2. Open Atom in Applications
3. Use `File -> Open` and open `atom_test`
4. You should see this:

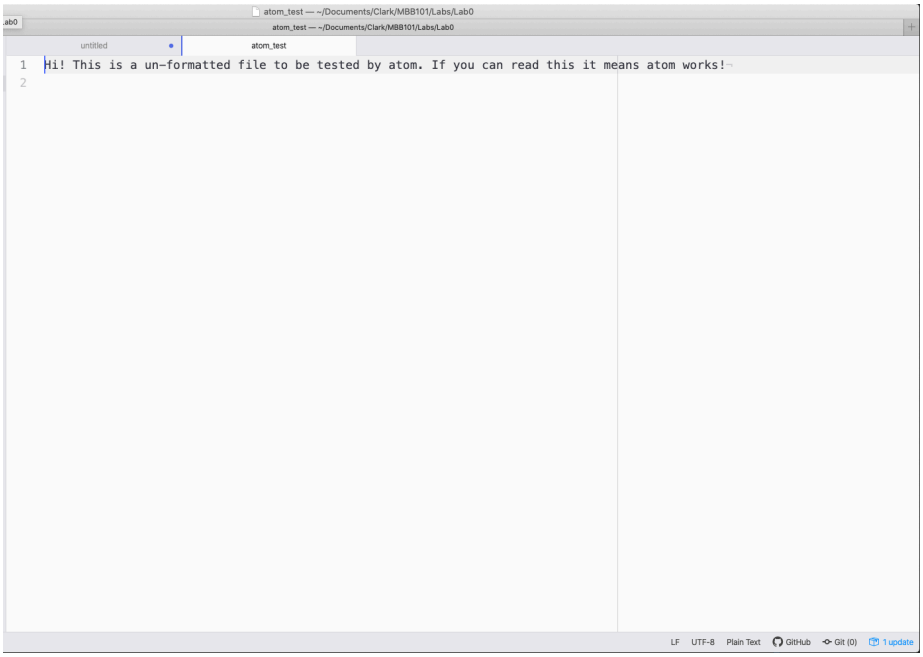



Figure 4.3: RStudio panels

We will use Atom to open more of these kinds of files that have no programs associated with them! Atom can open any text file that is unformatted, as well as many scripts and programs. We won't go so deep into these other elements (Remember, this is an *Introduction* course!) but we can discuss them in the lab.

# Chapter 5

# Introduction to electronic notebooks in R Markdown

The electronic lab notebooks we will use will in class will be created as `R Markdown documents`.

Markdown is a simple formatting syntax for authoring HTML, PDF, and MS Word documents. For more details on using R Markdown see http://rmarkdown.rstudio.com.

When you click the **Knit** button a document will be generated that includes both content as well as the output of any embedded R code chunks within the document.

## 5.1 Markdown formats

As mentioned above, Markdown is a simple formatting syntax for authoring HTML, PDF, and MS Word documents.

We will use R Markdown as it will allow us to create documents that we can modify in different computers (as long as we have R and R Studio).

We will call these documents *Electronic Lab Notebooks (ELS)*. These ELS' will be used to write down what you have done in class, save the results and the commands and also will tell us where the files you are using will be at.

But, before we go into the super complex part of it, lets learn the basics of R markdown.

## 5.2 Basics of Markdown

So, the idea to use the R markdown file as a E-notebook we need to understand what the syntax is.

> Syntax: a set of rules for or an analysis of the syntax of a language

### 5.2.1 Basic syntax

In the case of markdown, all the text that is not formatted in any way will be displayed in the most basic font.

So you'll see that all of these sentences are in simple format.

In addition, if you surround words with certain elements, it can become a word in **bold** or in *italics*.

In this case, when I add a `**` before and after the word I want to highlight, the word becomes a bold word. When I add a `**` before and after the word I want to highlight, the word becomes a word in italics.

If you want to add a header, or a subtitle, start a sentence with `#`. One `#` means main header, two will be a subtitle, and so on.

### 5.2.2 Lines of code

Markdown allows you to create unformatted lines, or lines of code. In these lines you can add the code you are using for certain steps, or to illustrate examples.

To create a code in-line, just add a ‘`(backtick)` between the line of code

#### 5.2.2.1 Example of inline code:

Similarity searches were done using `blastn` with default parameters and an expected E-value of 10-5.

#### 5.2.2.2 Example of code blocks:

To create a code block, start a region with three ‘`(backticks)`. Add the code in the line after the backticks and then close with three additional backticks.

```
This is a code-block
blastp -query q
```

### 5.2.3 Lists

You can create unordered or ordered lists in Markdown. To create **unordered** lists in markdown, start each element of the list with a - sign:

**Code:**

```
- Bananas
- Pijamas
- Bandanas
```

**Result:**

- Bananas
- Pijamas
- Bandanas

To create ordered lists, add a number with a period before each element. The numbers don't have to be consecutive, which will allow Markdown to modify the list no problem *(Check the raw code before you run the Knitr command)*:

**Code:**

```
1. Wake up
1. Make the bed
1. Lie down again
```

**Result:**

1. Wake up
2. Make the bed
3. Lie down again

You can also make nested lists! Nested lists can are done by creating list using **4** spaces:

**Code:**

```
1. This week:
    - Homework
    - Breakfast
    - Cats litter
2. This weekend
    - Nothing
```